



# Grafana Loki: *Like Prometheus, but for logs.*

Tom Wilkie, Feb 2019





**Tom Wilkie** *VP Product, Grafana Labs*

*Previously: Kausal, Weaveworks, Google, Acunu, Xensource*

Prometheus & Cortex maintainer, mixins authors etc

Twitter: *@tom\_wilkie*    Email: *tom@grafana.com*

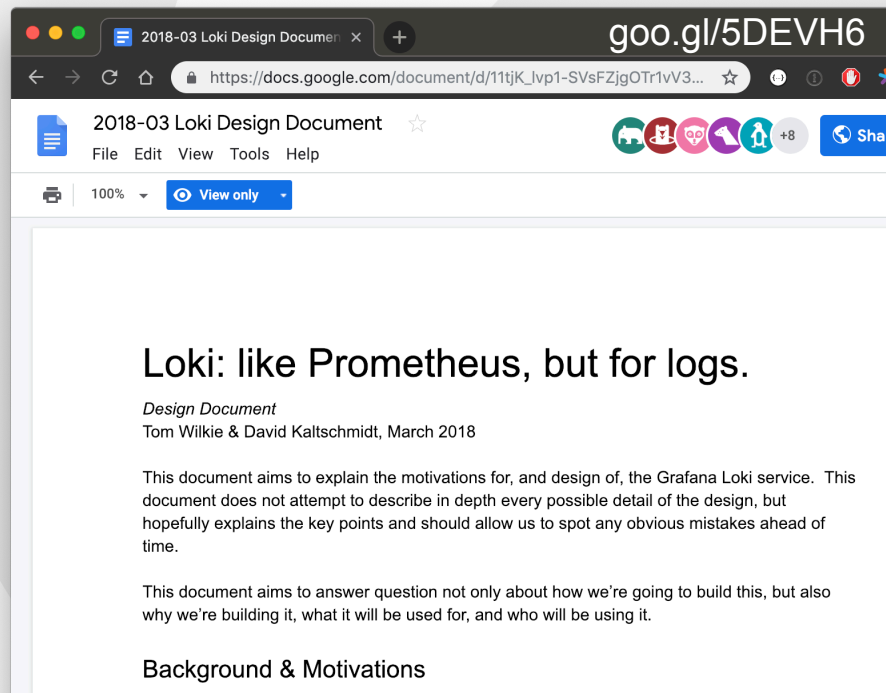




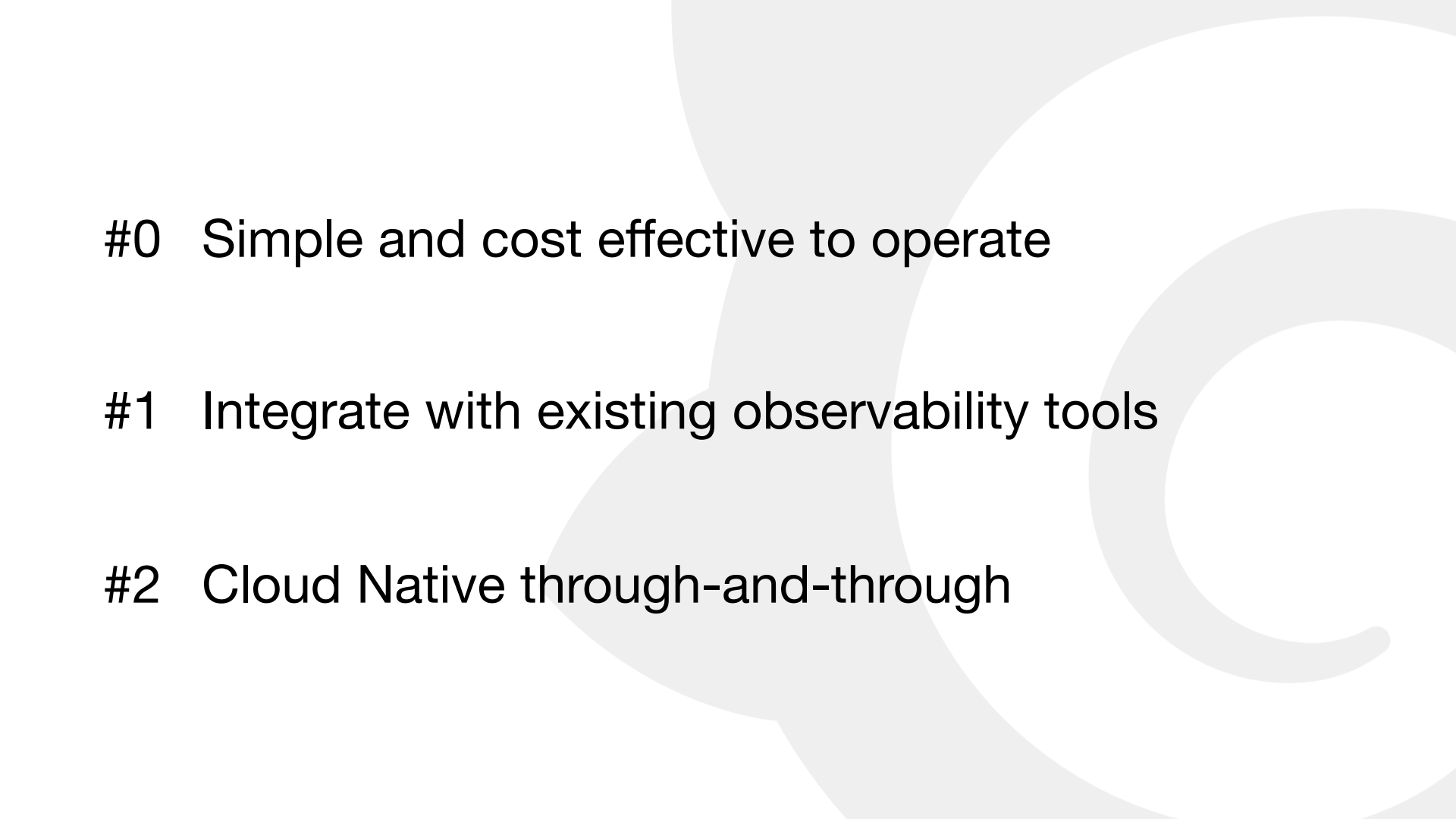
*Loki is a horizontally-scalable, highly-available, multi-tenant log aggregation system inspired by Prometheus.*

03/18 Project started  
12/18 Launched at KubeCon  
12/18 #1 on HN for ~12hrs!  
01/19 ~5k GitHub stars

<https://github.com/grafana/loki>







#0 Simple and cost effective to operate

#1 Integrate with existing observability tools

#2 Cloud Native through-and-through

*#0 Simple to scale*

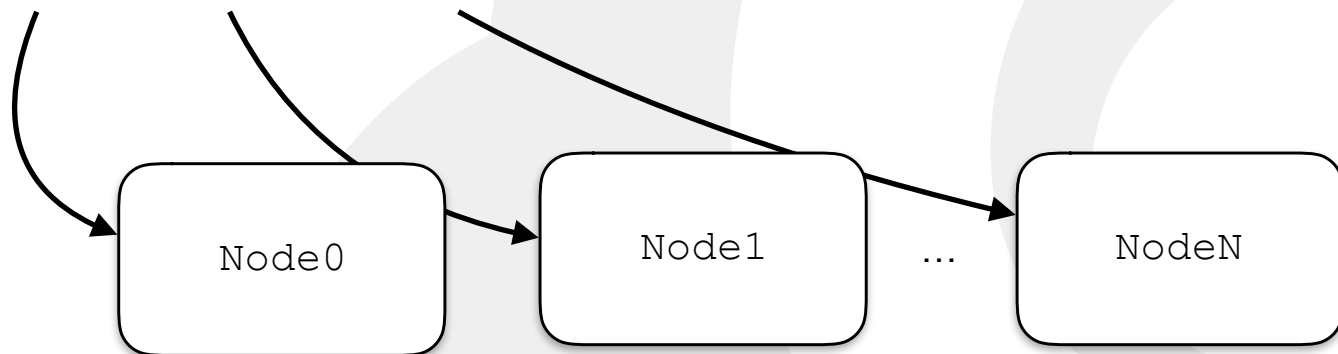
```
DEwMGIwZ => {  
  time: "2018-01-31 15:41:04",  
  job: "frontend",  
  env: "dev",  
  line: "POST /api/prom/push..."  
}
```



|                                 |               |
|---------------------------------|---------------|
| ("time", "2018-01-31 15:41:04") | -> "DEwMGIwZ" |
| ("job", "frontend")             | -> "DEwMGIwZ" |
| ("env", "dev")                  | -> "DEwMGIwZ" |
| ("line", "POST")                | -> "DEwMGIwZ" |
| ("line", "/api/prom/push")      | -> "DEwMGIwZ" |
| ("line", "HTTP/1.1")            | -> "DEwMGIwZ" |
| ("line", "502")                 | -> "DEwMGIwZ" |

*Existing log aggregation systems do full text indexing and support complex queries*

|                                 |               |
|---------------------------------|---------------|
| ("time", "2018-01-31 15:41:04") | -> "DEwMGIwZ" |
| ("job", "frontend")             | -> "DEwMGIwZ" |
| ("env", "dev")                  | -> "DEwMGIwZ" |
| ("line", "POST")                | -> "DEwMGIwZ" |
| ("line", "/api/prom/push")      | -> "DEwMGIwZ" |
| ("line", "HTTP/1.1")            | -> "DEwMGIwZ" |
| ("line", "502")                 | -> "DEwMGIwZ" |



*Existing log aggregation systems do full text indexing and support complex queries*

```
{job="frontend", env="dev"} => {  
  time:  "2018-01-31 15:41:04",  
  line:  "POST /api/prom/push HTTP/1.1 502 0"  
}
```

*Loki doesn't index the text of the logs, instead grouping entries into "streams" and indexing those with labels.*

*#1 Integrated with  
existing tools*



**The Watchmen** APP 5:42 PM

[FIRING:3] CFRoutesTooLow (cf router pcf-dev 10.130.7.229:9186 pcf-dev-firehose codelab-monitor gorouter)

The number of routes in the router's routing table is dangerously low: 113

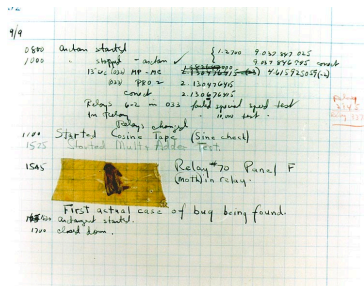
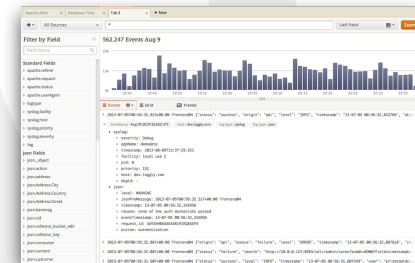


**The Watchmen** APP 5:54 PM

[RESOLVED] CFRoutesTooLow (cf router pcf-dev 10.130.7.229:9186 pcf-dev-firehose codelab-monitor gorouter)

The number of routes in the router's routing table is dangerously low: 113

The screenshot shows the 'Test' application interface. At the top, there's a 'Test results' window. The main part of the window displays a 'Top query metrics view (seconds)' graph. Below this, there are several smaller graphs arranged in a grid. The graphs show various metrics over time, including traffic, response time, new user sessions, new board, errors, and positive query pages view. Each graph has a legend and a title. The x-axis for all graphs represents time, ranging from 00:00:00 to 00:00:00. The y-axis represents the value of the metric being measured.

[illegible]

## 5. Distributed Tracing

## 4. Log Aggregation





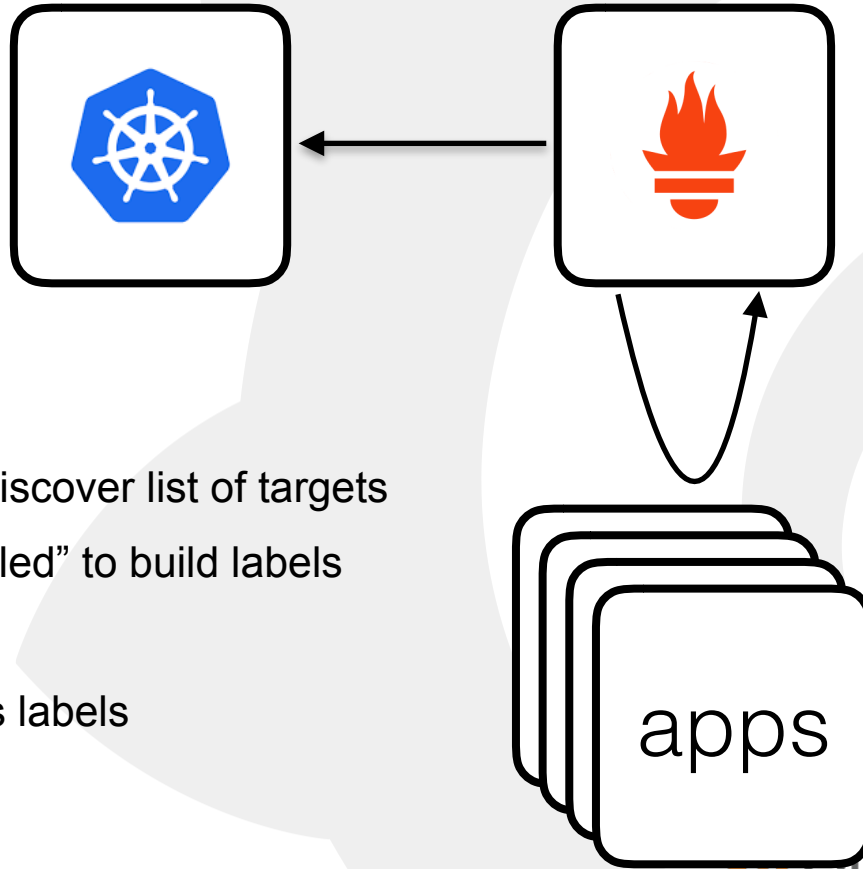
Prometheus' data model is very simple:

`<identifier> → [ (t0, v0), (t1, v1), ... ]`

Timestamps are millisecond int64, values are float64

Identifiers are bags of (label, value) pairs:

`{job="foo", instance="bar", ... }`



#0 Prometheus talks to k8s to discover list of targets

#1 Target information is “relabelled” to build labels

#2 Metrics are pulled from apps

#3 Target labels added to series labels

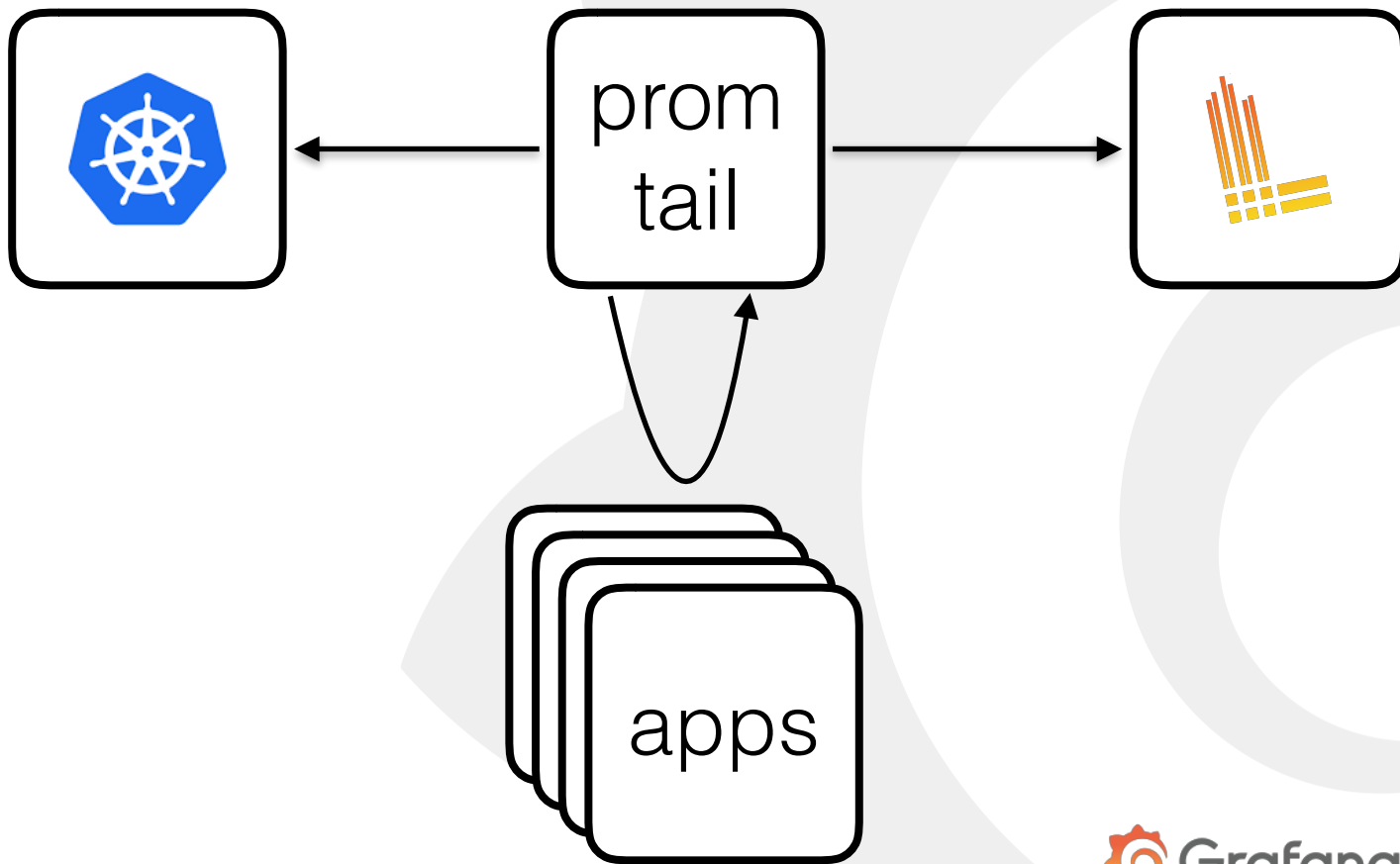


Loki's data model is very similar:

```
<identifier> → [ (t0, v0), (t1, v1), ... ]
```

Timestamps are nanosecond floats, values are byte arrays.

Identifiers are the same - label sets.



rate(job="tempo-dev/promtail",\_\_name\_\_="http\_requests\_total")[5m]

Graph



```
(code="200"/>
handler="prometheus" instance="promtail-7arsw" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(code="200"/>
handler="prometheus" instance="promtail-824n8" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-99cov" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-cf9d4" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-cp87n" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-d87k" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-d91qj" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-g9fnd" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-g25w" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-gkzr" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-h9fj" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-k0zr" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-l05d" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-r02m" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-vu7n" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-w83q" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-wp45" job="tempo-dev/promtail" method="get" namespace="tempo-dev"
(handler="prometheus" instance="promtail-x4qg" job="tempo-dev/promtail" method="get" namespace="tempo-dev")
```

Table

| code | handler    | instance       | job                | method | namespace | Value #1           |
|------|------------|----------------|--------------------|--------|-----------|--------------------|
| 00   | prometheus | promtail-7arsw | tempo-dev/promtail | get    | tempo-dev | 0.2                |
| 00   | prometheus | promtail-824n8 | tempo-dev/promtail | get    | tempo-dev | 0.200000701766...  |
| 00   | prometheus | promtail-99cov | tempo-dev/promtail | get    | tempo-dev | 0.2                |
| 00   | prometheus | promtail-cf9d4 | tempo-dev/promtail | get    | tempo-dev | 0.2                |
| 00   | prometheus | promtail-cp87n | tempo-dev/promtail | get    | tempo-dev | 0.1929973541817... |
| 00   | prometheus | promtail-d87k  | tempo-dev/promtail | get    | tempo-dev | 0.1930061586510... |

Log labels (job="tempo-dev/promtail")

Search by regular expression

Logs



Timestamp Local time Labels Dedup none exact numbers signature

Common labels: tempo-dev/promtail tempo-dev stderr Unit: 1000 (20 returned)

```
2018-12-28 12:14:58 level=info ts=2018-12-28T11:14:58.269487262Z caller=target.go:179 msg="stopping tailing file"
=var/log/pods/336c2c25-89e5-11e9-a5e5-42818a96802e/querier/0.log
2018-12-28 12:14:58 level=info ts=2018-12-28T11:14:58.267611813Z caller=targetmanager.go:155 msg="Removing target
instance=\"querier-649df68b47-fkrvt\", job=\"tempo-dev/querier\", namespace=\"tempo-dev\"}"
2018-12-28 12:14:58 level=info ts=2018-12-28T11:14:58.214648517Z caller=target.go:179 msg="stopping tailing file"
=var/log/pods/336c2c25-89e5-11e9-a5e5-42818a96802e/querier/1.log
2018-12-28 11:14:57 level=info ts=2018-12-28T11:14:57.481582287Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/cc6ded8-8a91-11e9-a5e5-42818a96802e/querier/0.log
2018-12-28 12:14:57 level=info ts=2018-12-28T11:14:57.481254963Z caller=targetmanager.go:142 msg="Adding target"
tance=\"querier-649df68b47-w4fd\", job=\"tempo-dev/querier\", namespace=\"tempo-dev\"}"
2018-12-28 12:14:00 level=info ts=2018-12-28T11:14:00.942599185Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/ab975435-8a91-11e9-a5e5-42818a96802e/querier/0.log
2018-12-28 12:14:00 level=info ts=2018-12-28T11:14:00.941896884Z caller=targetmanager.go:142 msg="Adding target"
tance=\"querier-649df68b47-hu3ph\", job=\"tempo-dev/querier\", namespace=\"tempo-dev\"}"
2018-12-28 12:13:59 level=info ts=2018-12-28T11:13:59.489257887Z caller=target.go:179 msg="stopping tailing file"
=var/log/pods/36a7631c-89e7-11e9-a5e5-42818a96802e/querier/0.log
2018-12-28 12:13:59 level=info ts=2018-12-28T11:13:59.488663625Z caller=targetmanager.go:155 msg="Removing target
instance=\"querier-649df68b47-jhsbb\", job=\"tempo-dev/querier\", namespace=\"tempo-dev\"}"
2018-12-28 11:13:59 level=info ts=2018-12-28T11:13:59.261927608Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/81fcaf8-8487-11e9-a2af-42818a968133/prometheus/1.log
2018-12-28 11:13:41 level=info ts=2018-12-28T11:13:41.157358245Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/8b8ac56-8a1e-11e9-a5e5-42818a96802e/querier/1.log
2018-12-28 12:13:39 level=info ts=2018-12-28T11:13:39.457358245Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/8b8ac56-8a1e-11e9-a5e5-42818a96802e/querier/1.log
2018-12-28 11:13:39 level=info ts=2018-12-28T11:13:39.457358245Z caller=target.go:182 msg="start tailing file" f
ar/log/pods/8b8ac56-8a1e-11e9-a5e5-42818a96802e/querier/1.log
```



**The Watchmen** APP 5:42 PM

[FIRING:3] CFRoutesTooLow (cf router pcf-dev 10.130.7.229:9186 pcf-dev-firehose codelab-monitor gorouter)

The number of routes in the router's routing table is dangerously low: 113



**The Watchmen** APP 5:54 PM

[RESOLVED] CFRoutesTooLow (cf router pcf-dev 10.130.7.229:9186 pcf-dev-firehose codelab-monitor gorouter)

The number of routes in the router's routing table is dangerously low: 113

The screenshot displays the Cortex-M4 demo application interface. At the top, there is a navigation bar with the text "Cortex / Wlites -" and a toolbar with icons for Home, Up, Down, and Help. Below the navigation bar, there are four main sections: "486K reqps", "11.37 Mil", "FPS", and "4.88K reqps". The "FPS" section is currently selected, showing a chart with a green area representing the FPS over time. The chart has a y-axis ranging from 0 to 0.015 and an x-axis ranging from 0 to 1000. Below the chart, there is a legend indicating "FPS" and "Latency". The "Latency" section shows a similar chart with a y-axis ranging from 0 to 1000 and an x-axis ranging from 0 to 1000. The legend indicates "FPS" and "Latency". The bottom of the interface features a "Distributor" section with a similar FPS and Latency chart.

The screenshot shows a Grafana dashboard with the following components:

- Header:** Includes the Grafana logo, a search bar with the text "prometheus", and navigation links for "Alerts", "Last 5 hours", "Clear All", and "Run Query".
- Query Editor:** Displays the query `sum(rate(cortex_request_duration_seconds_count{tikv}) by [job])`. It also shows the data source as "Prometheus" and the time range as "Last 5 hours".
- Graph:** A line graph showing the request duration over time. The y-axis represents the count, ranging from 0 to 8000. The x-axis represents time, from 15:00 to 17:00. The graph shows a steady decline in the request duration, with a sharp drop around 16:00, followed by a recovery. A legend at the bottom identifies the data series as `job=cortex-ctlupgrade`.
- Table:** A table with two columns: "job" and "Value #1". It lists the following jobs and their corresponding values:
 

| job               | Value #1           |
|-------------------|--------------------|
| cortex-ctlupgrade | 1.7777777777777777 |
| cortex-ctlupgrade | 0.5                |
| cortex-ctlupgrade | 90.5237997460791   |
| cortex-ctlupgrade | 4473.547777777778  |
| cortex-ctlupgrade | 2.4777777777777777 |

9/9

8800 Autos started  
1000 stopped - auto ✓ { 1200 7:00 100 0.5  
1500 1500 AT-MS ~~1200 100 0.5~~ 7:00 1500 7:00 0.5  
DIN 7:00 2 1200 1000  
Circuit 1200 1000  
Relay 600 m. 0.52 full signal used test  
in relay - none sent.

11:00 Started Casino Tape (Sine chail)  
15:25 Started Casino Ticker Test.

15:45



Relay 70 Panel F  
Autos in relay.

16:50 First actual case of bug being found.  
Autos started  
17:00 closed down.

[illegible]

The screenshot shows the Databricks workspace interface. At the top, there's a navigation bar with 'Explore' and 'Legacy' tabs. Below that, a histogram titled 'log(metrics/cassandra)' displays the distribution of log values. The x-axis ranges from -10 to 10, and the y-axis (frequency) ranges from 0 to 15. The data is represented by orange bars. Below the histogram, a table of command logs is visible, showing commands executed on the cluster.

| Timestamp           | Command ID                              | Command Text                                                                                                                                                           |
|---------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2019-10-16 18:19:21 | exp@cs5000wvny, controller_revison_bash | command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra) |
| 2019-10-16 18:19:21 | exp@cs5000wvny, controller_revison_bash | command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra) |
| 2019-10-16 18:19:21 | exp@cs5000wvny, controller_revison_bash | command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra) |
| 2019-10-16 18:19:21 | exp@cs5000wvny, controller_revison_bash | command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra), command@cs5000wvny-efs-0007897, log(metrics/cassandra) |

## 4. Log Aggregation

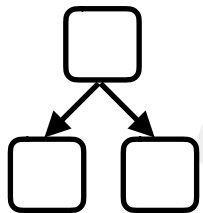
# *#2 Cloud Native*



containerised



Kubernetes Native

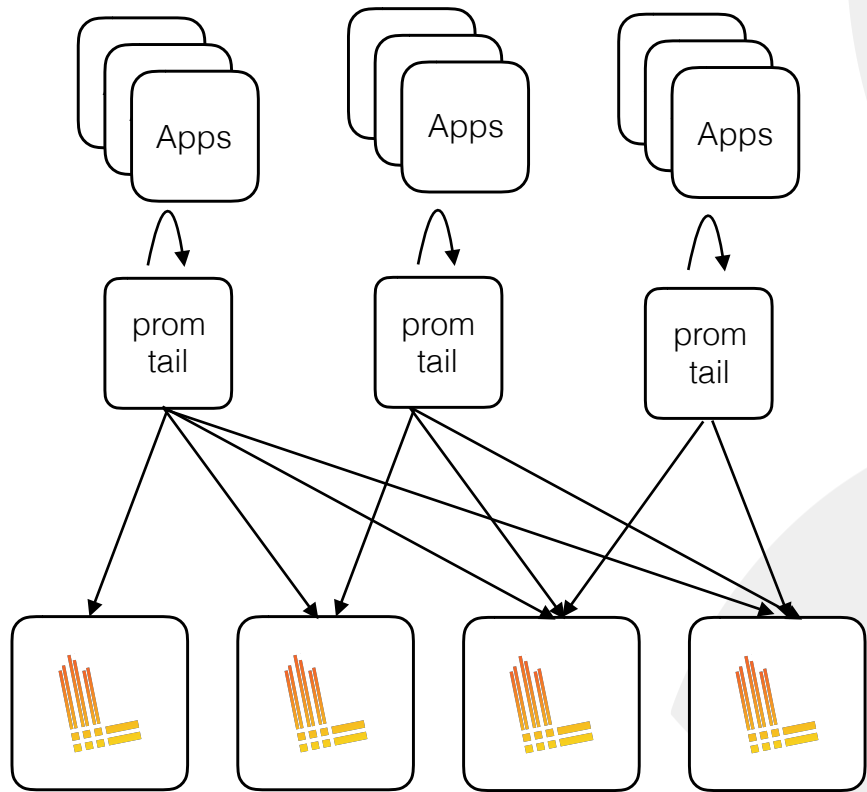


(optionally)  
Microservices

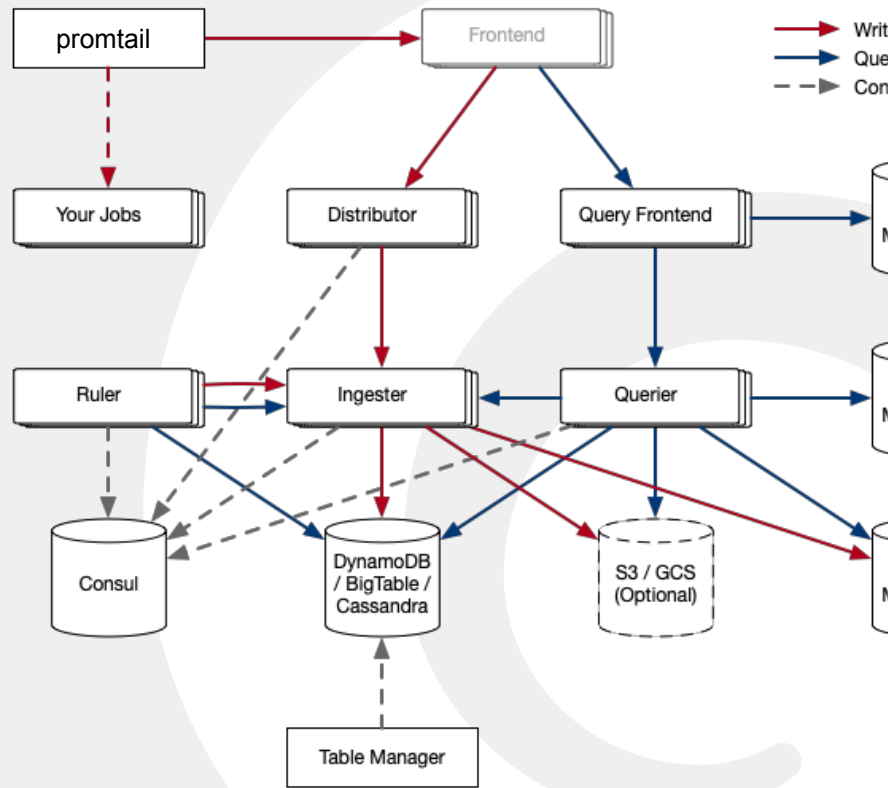


Cloud Storage

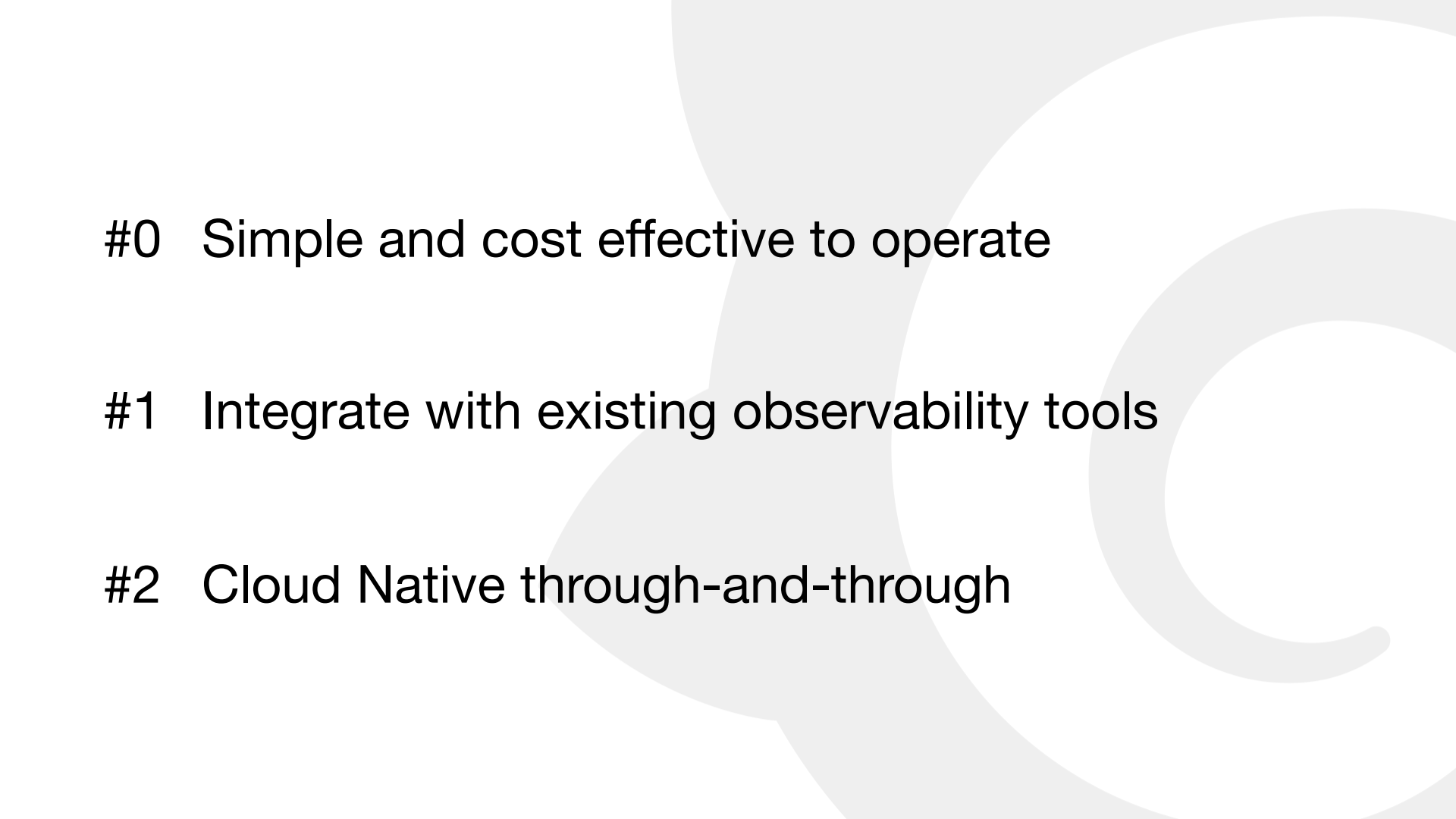




*(optionally) microservices*



*Demo(?)*



#0 Simple and cost effective to operate

#1 Integrate with existing observability tools

#2 Cloud Native through-and-through

*Thanks! Questions?*

# Grafana Cloud:

Your complete, fully managed, hosted metrics platform.



Grafana Cloud

**Store, visualize and alert without the headache of scaling or managing your own monitoring stack.**

## What is Grafana Cloud?

Grafana Cloud is a hosted and fully managed SaaS metrics platform that helps Ops and Dev teams using Grafana to understand the behavior of their applications and infrastructure

Grafana Cloud allows users to provision and manage the best open source observability tools - Grafana and Prometheus - all through a simple UI and single API.



# Grafana



# Prometheus